

BORAMA HARDWARE

Implementation Build Plan

Phase 0: Init • Phase 1: Database • Phase 2: Foundation

Phase 3: UI • Phase 4: Auth • Phases 5-13: Features • Phase 14: Deploy

A complete sequenced build guide for Claude Code — from first command to live deployment

Version 1.0 • July 2025 • Confidential

How to use this document:

Execute each phase in strict sequence. Do not start a phase until all checkpoints from the previous phase pass. Every step references source documents: [TECH] = Tech Stack Doc, [SCHEMA] = Backend Schema Doc, [GUIDE] = Content Guidelines Doc, [FLOW] = App Flow Doc, [PRD] = Product Requirements Doc. Steps tagged [DB] run in the Supabase SQL Editor. Steps tagged [CLI] run in the terminal. Steps tagged [CODE] create or edit files in the project. Steps tagged [VERIFY] confirm a condition.

Phase	Name	Key Output	Effort
0	Project Initialization	Repo, dependencies, config files	~1 hr
1	Supabase Database	35 tables, functions, triggers, RLS, seed data	~2 hr
2	Core Infrastructure	Supabase clients, types, i18n, middleware, utilities	~2 hr
3	Global UI System	All reusable components per Guidelines doc	~4 hr
4	Authentication	Phone OTP flow, protected routes, auth store	~2 hr
5	Customer Store — Catalog	Home, categories, product detail, search pages	~4 hr
6	Customer Store — Orders	Cart, checkout, order tracking, loyalty, returns	~4 hr
7	Admin Portal	Product/order/staff/analytics/settings management	~5 hr

Phase	Name	Key Output	Effort
8	Inventory Portal	Stock management, stocktakes, alerts, forecasts	~3 hr
9	POS Terminal	Full POS with split pay, offline, session management	~5 hr
10	Staff Portal	Pickup, payment confirm, return processing	~2 hr
11	AI Features	Chatbot, estimator, semantic search, descriptions	~3 hr
12	PWA & Offline Mode	Service worker, offline queue, background sync	~2 hr
13	Testing	Vitest unit tests, Playwright E2E tests	~3 hr
14	Deployment	Vercel deploy, env vars, domain, go-live checklist	~1 hr

Phase 0 — Project Initialization

Goal: Create the Next.js project, install all pinned dependencies, configure all tooling files, and establish the full directory structure. Nothing should be installed without an exact version number.

Step 0.1 [CLI] — Verify runtime environment

```
# Verify Node.js 20.18.1 LTS (required exact version – see TECH doc)
node --version # must print v20.18.1

# If wrong version, use nvm:
nvm install 20.18.1
nvm use 20.18.1

# Enable pnpm via corepack (ships with Node.js 16.13+)
corepack enable
corepack prepare pnpm@9.12.1 --activate
pnpm --version # must print 9.12.1
```

Step 0.2 [CLI] — Scaffold Next.js project

```
# Create the project (answer prompts as shown below)
pnpm create next-app@14.2.15 borama-hardware --typescript --tailwind --eslint --app --src-dir
--import-alias '@/*'

# Prompts: TypeScript? YES | ESLint? YES | Tailwind? YES | src/ dir? YES
# App Router? YES | import alias? YES (@/*)

cd borama-hardware

# Create .nvmrc
echo '20.18.1' > .nvmrc
```

Step 0.3 [CLI] — Install all production dependencies (exact versions)

```
pnpm add \
next@14.2.15 react@18.3.1 react-dom@18.3.1 \
@supabase/supabase-js@2.45.4 @supabase/ssr@0.5.1 \
@google/generative-ai@0.21.0 \
@upstash/ratelimit@2.0.4 @upstash/redis@1.34.3 \
twilio@5.3.5 \
zustand@4.5.5 @tanstack/react-query@5.59.0 \
react-hook-form@7.53.1 zod@3.23.8 @hookform/resolvers@3.9.0 \
next-intl@3.22.1 next-pwa@5.6.0 \
@radix-ui/react-dialog@1.1.2 @radix-ui/react-dropdown-menu@2.1.2 \
@radix-ui/react-select@2.1.2 @radix-ui/react-toast@1.2.2 \
@radix-ui/react-tooltip@1.1.3 @radix-ui/react-tabs@1.1.1 \
@radix-ui/react-checkbox@1.1.2 @radix-ui/react-switch@1.1.1 \
@radix-ui/react-progress@1.1.0 @radix-ui/react-separator@1.1.0 \
@radix-ui/react-avatar@1.1.1 @radix-ui/react-badge@1.0.0 \
@radix-ui/react-popover@1.1.2 @radix-ui/react-slot@1.1.0 \
lucide-react@0.454.0 class-variance-authority@0.7.0 \
clsx@2.1.1 tailwind-merge@2.5.4 \
jspdf@2.5.2 papaparse@5.4.1 idb@8.0.0 react-barcode@1.5.3 \
```

```
sharp@0.33.5 date-fns@3.6.0 react-hot-toast@2.4.1
```

Step 0.4 [CLI] — Install all dev dependencies (exact versions)

```
pnpm add -D \
typescript@5.6.3 @types/react@18.3.11 @types/node@20.16.11 \
@types/react-dom@18.3.1 @types/papaparse@5.3.14 \
tailwindcss@3.4.14 postcss@8.4.47 autoprefixer@10.4.20 \
eslint@8.57.1 eslint-config-next@14.2.15 \
prettier@3.3.3 prettier-plugin-tailwindcss@0.6.8 \
vitest@2.1.3 @vitest/ui@2.1.3 \
@testing-library/react@16.0.1 @testing-library/jest-dom@6.5.0 \
@playwright/test@1.48.0 jsdom@25.0.1
```

Step 0.5 [CODE] — Create all configuration files

Create each file below exactly as specified in the **Tech Stack document**. Do not modify these files — the exact content is required for compatibility.

File Path	Purpose / Contents
tsconfig.json	Strict mode TS config — copy from Tech Stack doc Section 8.1
tailwind.config.ts	Orange brand theme, content paths, plugins — copy from Tech Stack doc Section 8.2
next.config.js	CommonJS format (NOT .ts) — PWA via next-pwa, image domains, i18n — Tech Stack doc Section 8.3
postcss.config.js	tailwindcss + autoprefixer — Tech Stack doc Section 8.4
.eslintrc.json	next/core-web-vitals config — Tech Stack doc Section 8.5
.prettierrc	Prettier config with tailwindcss plugin — Tech Stack doc Section 8.6
vitest.config.ts	jsdom environment, path aliases — Tech Stack doc Section 8.7
.env.local	All env variables listed in Schema doc Section 15.2 — NEVER commit this file
.env.example	Same keys as .env.local with empty values — commit this to git
.gitignore	next default plus: .env*.local, .vercel, *.tsbuildinfo

Step 0.6 [CODE] — Create directory structure

Create all directories exactly as shown. This is the full intended source tree. Create placeholder index.ts files in each directory so git tracks them.

```
src/
app/ # Next.js App Router root
(auth)/ # Route group — no layout
sign-in/page.tsx
verify/page.tsx
(customer)/ # Customer store portal
layout.tsx
page.tsx # Home page
products/
```

```
page.tsx # Product listing
[id]/page.tsx # Product detail
categories/[slug]/page.tsx
search/page.tsx
cart/page.tsx
checkout/page.tsx
orders/
page.tsx
[id]/page.tsx
loyalty/page.tsx
returns/
new/page.tsx
[id]/page.tsx
(admin)/ # Admin portal
layout.tsx
dashboard/page.tsx
products/
page.tsx
new/page.tsx
[id]/page.tsx
orders/page.tsx
staff/page.tsx
discounts/page.tsx
banners/page.tsx
locations/page.tsx
analytics/page.tsx
settings/page.tsx
(inventory)/ # Inventory portal
layout.tsx
dashboard/page.tsx
stock/page.tsx
receive/page.tsx
alerts/page.tsx
stocktakes/
page.tsx
[id]/page.tsx
forecasts/page.tsx
(pos)/ # POS terminal
layout.tsx
page.tsx # POS main screen
session/page.tsx
(staff)/ # Staff portal
layout.tsx
pickup/page.tsx
payments/page.tsx
returns/page.tsx
api/ # API routes
auth/
products/
orders/
returns/
loyalty/
ai/
admin/
inventory/
pos/
```

```
layout.tsx # Root layout
globals.css
components/
ui/ # Base design-system components
layout/ # Header, Footer, Sidebar, Nav
forms/ # Form field wrappers
customer/ # Customer portal specific
admin/ # Admin portal specific
inventory/ # Inventory portal specific
pos/ # POS terminal specific
staff/ # Staff portal specific
ai/ # AI chatbot, estimator
lib/
supabase/ # Client, server, admin instances
env.ts # Zod env validation
utils.ts # cn(), formatSLSH(), formatDate(), etc.
constants.ts # App-wide constants
ai.ts # Gemini client + helpers
sms.ts # Twilio helpers
ratelimit.ts # Upstash rate limit helpers
hooks/ # Custom React hooks
stores/ # Zustand stores
types/ # TypeScript type definitions
i18n/ # next-intl config + message files
middleware.ts # Auth + i18n middleware
messages/
en.json # English strings
so.json # Somali strings
public/
icons/ # PWA icons
manifest.json # PWA manifest
```

Step 0.7 [CLI] — Initialize git repository

```
git init
git add .
git commit -m 'chore: scaffold Next.js 14 project with all dependencies'
# Connect to GitHub (create repo on github.com first):
git remote add origin https://github.com/YOUR_ORG/borama-hardware.git
git branch -M main
git push -u origin main
```

CHECKPOINT — verify before proceeding:

- ✓ node --version prints v20.18.1
- ✓ pnpm --version prints 9.12.1
- ✓ pnpm dev runs without errors on localhost:3000
- ✓ All 7 config files exist and match Tech Stack doc
- ✓ .env.local exists with all required keys filled in
- ✓ Directory structure matches the tree above
- ✓ Git repo exists with initial commit pushed

Phase 1 — Supabase Database

Goal: Set up the complete PostgreSQL schema in Supabase — all 35 tables, extensions, indexes, functions, triggers, RLS policies, seed data, and storage buckets. The entire schema is specified in the **Backend Schema document**.

Prerequisites (must be complete before this phase):

→ Phase 0 complete — Supabase project created and URL + keys added to .env.local

Step 1.1 [DB] — Enable PostgreSQL extensions

Run in Supabase SQL Editor (Database > SQL Editor > New query):

```
-- Required extensions (run as first migration)
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_cron;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

Step 1.2 [DB] — Create Auth hook function

Create the JWT claim enrichment hook (Schema doc Section 2.3) and register it in Supabase Auth settings under 'Hook: Customize Access Token':

```
-- Creates the function from Schema doc Section 2.3
-- Then register in: Supabase Dashboard > Auth > Hooks > Customize Access Token
-- Set Function: auth.customize_access_token
```

Step 1.3 [DB] — Create RLS helper functions

Create auth.user_role(), auth.user_location_id(), auth.is_admin(), auth.is_staff() from Schema doc Section 2.4. These are used by all RLS policies.

Step 1.4 [DB] — Create all 35 tables (strict dependency order)

Create tables in the order below. Each table depends on the ones above it (foreign keys). Copy exact DDL for each table from Schema doc Sections 3-10.

Order	Table	Schema Section	Dependencies
1	locations	3	None
2	profiles	3	auth.users, locations
3	location_hours	3	locations
4	exchange_rates	3	profiles
5	mobile_money_settings	3	locations
6	banners	3	locations

Order	Table	Schema Section	Dependencies
7	categories	4	categories (self)
8	products	4	categories, profiles
9	product_images	4	products
10	product_variants	4	products
11	product_embeddings	4	products
12	inventory	5	products, product_variants, locations
13	stock_movements	5	products, product_variants, locations, profiles
14	inventory_alerts	5	products, product_variants, locations
15	loyalty_tiers	7	profiles
16	loyalty_cards	7	profiles
17	loyalty_transactions	7	loyalty_cards, profiles
18	discount_codes	6	profiles
19	orders	6	profiles, locations, discount_codes
20	order_items	6	orders, products, product_variants
21	discount_code_uses	6	discount_codes, profiles, orders
22	payment_transactions	6	orders
23	stocktakes	5	locations, profiles
24	stocktake_items	5	stocktakes, products, product_variants
25	pos_sessions	8	profiles, locations
26	pos_transactions	8	pos_sessions, locations, profiles, discount_codes
27	pos_transaction_items	8	pos_transactions, products, product_variants
28	pos_payment_splits	8	pos_transactions, profiles
29	parked_transactions	8	pos_sessions, profiles, locations
30	returns	9	orders, pos_transactions, profiles, locations
31	return_items	9	returns, order_items, products, product_variants
32	ai_forecasts	9	products, product_variants, locations
33	audit_log	10	profiles
34	sms_logs	10	None (text fields only)

Step 1.5 [DB] — Create indexes

```
-- Trigram indexes for fast keyword search
CREATE INDEX idx_products_name_en_trgm ON products USING GIN (name_en gin_trgm_ops);
CREATE INDEX idx_products_name_so_trgm ON products USING GIN (name_so gin_trgm_ops);
CREATE INDEX idx_products_tags ON products USING GIN (tags);

-- IVFFlat index for pgvector similarity search
-- NOTE: Create AFTER inserting at least 100 embeddings (lists=100 requires enough data)
-- CREATE INDEX idx_embeddings_ivfflat ON product_embeddings
```

```
-- USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
-- Performance indexes on frequently-queried columns
CREATE INDEX idx_orders_customer ON orders (customer_id);
CREATE INDEX idx_orders_location ON orders (location_id);
CREATE INDEX idx_orders_status ON orders (status);
CREATE INDEX idx_inventory_location ON inventory (location_id);
CREATE INDEX idx_pos_transactions_session ON pos_transactions (pos_session_id);
CREATE INDEX idx_audit_log_table ON audit_log (table_name, record_id);
```

Step 1.6 [DB] — Create all database functions

Run each function from Schema doc Section 11 in this order:

File Path	Purpose / Contents
Schema 11.1	generate_order_number() + trg_set_order_number trigger on orders
Schema 11.2	generate_pickup_code() — called manually from API routes
Schema 11.3	calculate_loyalty_tier(lifetime_points) — called by loyalty trigger
Schema 11.4	check_discount_code_validity() — called from checkout API
Schema 11.5	match_products_semantic() — called from /api/ai/search
Schema 11.6	release_expired_reservations() + pg_cron schedule (every 10 min)
Schema 11.7	expire_parked_transactions() + pg_cron schedule (every 30 min)
Schema 11.8	generate_pos_transaction_number() + trigger on pos_transactions

Step 1.7 [DB] — Create all triggers

Run each trigger function and CREATE TRIGGER statement from Schema doc Section 12 in order:

File Path	Purpose / Contents
Schema 12.1	trg_auto_create_profile — AFTER INSERT ON auth.users
Schema 12.2	trg_reserve_inventory — AFTER INSERT ON orders
Schema 12.3	trg_confirm_inventory — AFTER UPDATE OF status ON orders (payment_confirmed)
Schema 12.4	trg_release_reservation — AFTER UPDATE OF status ON orders (cancelled)
Schema 12.5	trg_create_loyalty_card — AFTER INSERT ON profiles (role = 'customer')
Schema 12.6	trg_award_loyalty_points — attached to both orders and pos_transactions
Schema 12.7	trg_check_low_stock — AFTER UPDATE OF quantity_on_hand ON inventory
Schema 12.8	trg_audit_log — applied to all 10 audited tables
Schema 12.9	trg_update_session_totals — AFTER INSERT OR UPDATE ON pos_transactions
Schema 12.10	trg_apply_stocktake — AFTER UPDATE OF status ON stocktakes (approved)

Step 1.8 [DB] — Enable RLS and create all policies

For every table: run ALTER TABLE ... ENABLE ROW LEVEL SECURITY, then create policies per the matrix in Schema doc Section 13. Use the sample SQL in Section 13 as a template.

Critical: Run ALTER TABLE ... ENABLE ROW LEVEL SECURITY on ALL 35 tables before creating policies. A table with RLS enabled but no policies will deny all access to non-service-role clients. Verify each table has at least one SELECT policy before moving on.

Step 1.9 [DB] — Insert seed data

```
-- Loyalty tiers (required for tier calculation to work)
INSERT INTO loyalty_tiers (tier_name, tier_name_so, min_lifetime_points, discount_percentage)
VALUES
('bronze', 'Naxaas', 0, 2.00),
('silver', 'Lacag', 500, 5.00),
('gold', 'Dahab', 2000, 10.00);

-- Default exchange rate (update to current rate)
-- (Insert after creating first admin profile via Supabase Auth)
-- INSERT INTO exchange_rates (usd_to_slsh_rate, set_by) VALUES (570.00, '<admin_user_id>');

-- Default location (Borama main branch)
INSERT INTO locations (name_en, name_so, address, phone)
VALUES ('Main Branch', 'Laan Hore', 'Borama City Center, Somaliland', '+252637000000');
```

Step 1.10 [DB] — Configure Supabase Auth + Storage

In Supabase Dashboard perform these configuration steps:

File Path	Purpose / Contents
Auth > Settings > Phone	Enable phone OTP. Set OTP expiry to 600 seconds. Enable 'Confirm phone number on signup'.
Auth > Settings > SMS Provider	Select Twilio. Enter Account SID, Auth Token, and phone number from .env.local.
Auth > Hooks	Register auth.customize_access_token as the 'Customize Access Token' hook.
Storage > New bucket	Create 6 buckets from Schema doc Section 15.1: product-images (public), product-thumbnails (public), category-icons (public), banners (public), return-photos (private), receipts (private)
Storage > Policies	For private buckets: authenticated users can upload to own folder. Authenticated can download own files.

CHECKPOINT — verify before proceeding:

- ✓ All 35 tables visible in Supabase Table Editor
- ✓ RLS enabled on every table (green shield icon visible)
- ✓ Functions visible in Database > Functions
- ✓ pg_cron jobs visible in Database > Extensions > pg_cron
- ✓ loyalty_tiers has 3 rows (bronze, silver, gold)
- ✓ locations has at least 1 row (Main Branch)
- ✓ 6 Storage buckets created with correct visibility
- ✓ Phone OTP test works: send OTP to your phone and verify
- ✓ Creating a test user via Auth creates a profiles row automatically

Phase 2 — Core Infrastructure

Goal: Create all foundational code modules — Supabase client instances, environment validation, TypeScript types, i18n configuration, utility helpers, Zustand stores, and Next.js middleware. No UI or business logic yet.

Prerequisites (must be complete before this phase):

- Phase 0 complete
- Phase 1 complete — Supabase is fully configured

Step 2.1 [CODE] — Environment validation (src/lib/env.ts)

Validate all environment variables at startup using Zod. The application throws at build time if any required env var is missing. Pattern from Tech Stack doc.

```
// src/lib/env.ts
import { z } from 'zod';

const envSchema = z.object({
  NEXT_PUBLIC_SUPABASE_URL: z.string().url(),
  NEXT_PUBLIC_SUPABASE_ANON_KEY: z.string().min(1),
  SUPABASE_SERVICE_ROLE_KEY: z.string().min(1),
  GEMINI_API_KEY: z.string().min(1),
  UPSTASH_REDIS_REST_URL: z.string().url(),
  UPSTASH_REDIS_REST_TOKEN: z.string().min(1),
  NEXT_PUBLIC_APP_URL: z.string().url(),
  NEXT_PUBLIC_DEFAULT_LOCATION_ID: z.string().uuid(),
});

export const env = envSchema.parse(process.env);
```

Step 2.2 [CODE] — Supabase client instances

File Path	Purpose / Contents
src/lib/supabase/client.ts	Browser client using @supabase/ssr createBrowserClient. Used in client components and hooks.
src/lib/supabase/server.ts	Server client using @supabase/ssr createServerClient with cookies(). Used in Server Components and Route Handlers.
src/lib/supabase/admin.ts	Service role client using createClient from @supabase/supabase-js + SUPABASE_SERVICE_ROLE_KEY. Used ONLY in API routes. Never import in client-side code.
src/lib/supabase/types.ts	Run: pnpm supabase gen types typescript --project-id YOUR_PROJECT_ID > src/lib/supabase/types.ts (generates full type-safe DB schema)

Step 2.3 [CODE] — TypeScript types (src/types/)

File Path	Purpose / Contents
src/types/database.ts	Re-export Database type from supabase/types.ts. Define Table<T> and Row<T> helpers.

File Path	Purpose / Contents
src/types/auth.ts	UserRole type: 'customer' 'cashier' 'inventory_manager' 'admin'. AuthUser interface with profile fields.
src/types/product.ts	ProductWithVariants, ProductVariant, ProductImage, Category, SearchResult interfaces.
src/types/order.ts	Order, OrderItem, OrderStatus, PaymentMethod, PaymentStatus, CartItem interfaces.
src/types/pos.ts	PosSession, PosTransaction, PosTransactionItem, PaymentSplit, ParkedCart interfaces.
src/types/inventory.ts	InventoryLevel, StockMovement, InventoryAlert, Stocktake, StocktakeItem, AiForecast interfaces.
src/types/loyalty.ts	LoyaltyCard, LoyaltyTier, LoyaltyTransaction interfaces.
src/types/api.ts	ApiResponse<T>, ApiError, PaginatedResponse<T> generic types for all API routes.

Step 2.4 [CODE] — Utility helpers (src/lib/utls.ts)

```
// src/lib/utls.ts
import { clsx, type ClassValue } from 'clsx';
import { twMerge } from 'tailwind-merge';
// Tailwind class merging utility
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
// Format SLASH amount: 125000 -> '125,000 SLASH'
export function formatSLASH(amount: number): string {
  return new Intl.NumberFormat('en-SO').format(amount) + ' SLASH';
}
// Convert SLASH to USD using exchange rate
export function slshToUsd(slsh: number, rate: number): string {
  return '$' + (slsh / rate).toFixed(2);
}
// Format date: ISO string -> 'Jul 12, 2025'
export function formatDate(iso: string, locale = 'en'): string { ... }
// Generate initials from full name: 'Ahmed Ali' -> 'AA'
export function getInitials(name: string): string { ... }
// Truncate text with ellipsis
export function truncate(text: string, max: number): string { ... }
```

Step 2.5 [CODE] — Internationalization (next-intl)

File Path	Purpose / Contents
src/i18n/config.ts	next-intl config: locales=['en','so'], defaultLocale='en', timeZone='Africa/Nairobi'
src/i18n/request.ts	getRequestConfig() — loads messages/[locale].json from disk
messages/en.json	All English UI strings organized by namespace: common, auth, products, cart, orders, pos, admin, inventory, loyalty, ai

File Path	Purpose / Contents
messages/so.json	All Somali translations for the same keys (source: App Flow doc copy guidelines + content guidelines)
middleware.ts	Combine next-intl middleware with Supabase auth session refresh (see Step 2.6)

Step 2.6 [CODE] — Next.js middleware (middleware.ts)

```
// middleware.ts
import { createServerClient } from '@supabase/ssr';
import { NextResponse, type NextRequest } from 'next/server';

export async function middleware(request: NextRequest) {
  // 1. Refresh Supabase session cookie (required by @supabase/ssr)
  const response = NextResponse.next({ request });
  const supabase = createServerClient(/* env vars, cookies */);
  const { data: { user } } = await supabase.auth.getUser();

  // 2. Protected route guards (check pathname prefix):
  // /admin/* -> require role = 'admin'
  // /inventory/* -> require role IN ('inventory_manager', 'admin')
  // /pos/* -> require role IN ('cashier', 'inventory_manager', 'admin')
  // /staff/* -> require role IN ('cashier', 'inventory_manager', 'admin')
  // /orders/*, /loyalty/*, /returns/* -> require any authenticated user

  // 3. If not authorized: redirect to /sign-in?next=<current_path>
  return response;
}

export const config = {
  matcher: ['/(?!_next/static|_next/image|favicon.ico|api/auth).*'],
};
```

Step 2.7 [CODE] — Zustand stores (src/stores/)

File Path	Purpose / Contents
src/stores/auth.store.ts	Holds: user, profile, role. Actions: setUser(), clearUser(). Persisted to localStorage.
src/stores/cart.store.ts	Holds: items[], appliedCode, loyaltyRedemption. Actions: addItem, removeItem, updateQty, applyCode, clearCart. Persisted to localStorage + idb for offline.
src/stores/pos.store.ts	Holds: activeSession, activeCart, parkedCarts, activeCustomer. Actions for POS transaction lifecycle. Uses idb for offline queue.
src/stores/currency.store.ts	Holds: exchangeRate (number), displayCurrency ('SLSH' 'USD'). Actions: setRate(), toggleCurrency(). Fetches rate from /api/public/exchange-rate on mount.
src/stores/ui.store.ts	Holds: sidebar open/closed, active language. No persistence.

Step 2.8 [CODE] — External service clients

File Path	Purpose / Contents
<code>src/lib/ai.ts</code>	Initialize GoogleGenerativeAI with GEMINI_API_KEY. Export geminiPro (gemini-1.5-pro), geminiFlash (gemini-1.5-flash). Export embedText(text: string): Promise<number[]> using text-embedding-004 model.
<code>src/lib/ratelimit.ts</code>	Initialize @upstash/ratelimit. Export rateLimiters: apiGeneral (20 req/10s), aiChat (5 req/min), search (30 req/min), auth (5 req/5min).
<code>src/lib/sms.ts</code>	Initialize Twilio client. Export sendSms(to: string, body: string): Promise<void>. Logs to sms_logs table via admin client.
<code>src/lib/constants.ts</code>	Export: PAYMENT_METHODS array, ORDER_STATUSES array, USER_ROLES array, POINTS_PER_SLSH = 0.001 (1 point per 1000 SLSH), RESERVATION_TIMEOUT_MINUTES = 30, PARKED_CART_EXPIRY_HOURS = 4.

Step 2.9 [CODE] — Root layout (src/app/layout.tsx)

```
// src/app/layout.tsx
// - Import globals.css
// - Set up next-intl NextIntlClientProvider
// - Set up @tanstack/react-query QueryClientProvider
// - Set up react-hot-toast Toaster (position: top-right, orange theme)
// - Set metadata: title='Borama Hardware', description from PRD
// - Set viewport: width=device-width, initialScale=1, themeColor=#F97316
// - Link PWA manifest: /manifest.json
// - Font: use next/font with system fonts (no Google Fonts for offline support)
// - Body classes: min-h-screen bg-stone-50 text-stone-900 antialiased
```

CHECKPOINT — verify before proceeding:

- ✓ pnpm dev starts without TypeScript errors
- ✓ Supabase client connects: no 401 errors in browser console
- ✓ Middleware redirects /admin to /sign-in when unauthenticated
- ✓ env.ts throws descriptive error if GEMINI_API_KEY is removed from .env.local
- ✓ Zustand cart store persists items across page refreshes
- ✓ next-intl locale switching works (en <-> so in URL)

Phase 3 — Global UI System

Goal: Build every reusable UI component defined in the **Content Guidelines document**. All design decisions — colours, typography, spacing, motion, copy — are fully specified there. Claude Code must not make any design decision not covered by that document. Build components in dependency order (primitives first, composites last).

Prerequisites (must be complete before this phase):
→ Phase 2 complete — all utility helpers and stores available

Step 3.1 [CODE] — Global CSS tokens (src/app/globals.css)

Define CSS custom properties for all design tokens from Guidelines doc Sections 2-4. All colour values, font sizes, and spacing must match exactly.

```
/* src/app/globals.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    /* Brand colours from Guidelines Section 2 */
    --color-primary: 249 115 22; /* #F97316 */
    --color-primary-dark: 234 88 12; /* #EA580C */
    /* ... all other tokens from Guidelines doc */
  }
  * { @apply border-stone-200; }
  body { @apply bg-stone-50 text-stone-900; }
}
```

Step 3.2 [CODE] — Base primitive components (src/components/ui/)

Each component wraps a Radix UI primitive. Props, variants, and visual appearance are specified exactly in Guidelines doc Section 8 (Components).

File Path	Purpose / Contents
ui/button.tsx	Variants: primary (orange), secondary (outline), ghost, destructive (red), loading state. Sizes: sm, md, lg, icon. Uses CVA for variant logic.
ui/input.tsx	Text input with label, error message, helper text, left/right icon slots. Orange focus ring. Error state turns border red.
ui/textarea.tsx	Same as input but multiline. Character count display when maxLength set.
ui/select.tsx	Wraps Radix Select. Orange chevron icon. Supports placeholder, error, disabled states.
ui/checkbox.tsx	Wraps Radix Checkbox. Orange when checked. Supports label and disabled state.
ui/switch.tsx	Wraps Radix Switch. Orange when enabled. With label.

File Path	Purpose / Contents
<code>ui/badge.tsx</code>	Status badge from Guidelines Section 8.5 badge table — 18 variants covering all order/stock/role/loyalty/payment statuses.
<code>ui/card.tsx</code>	Card, CardHeader, CardTitle, CardDescription, CardContent, CardFooter. White bg, border, shadow-sm.
<code>ui/dialog.tsx</code>	Wraps Radix Dialog. Animated overlay (fade) and content (slide-up per Guidelines motion spec).
<code>ui/toast.tsx</code>	Wraps react-hot-toast with branded styles. Types: success (green), error (red), info (blue), loading (orange spinner).
<code>ui/tooltip.tsx</code>	Wraps Radix Tooltip. Dark bg, 200ms delay.
<code>ui/progress.tsx</code>	Linear progress bar. Orange fill. Animated width transition.
<code>ui/separator.tsx</code>	Horizontal/vertical divider using Radix Separator.
<code>ui/avatar.tsx</code>	Wraps Radix Avatar. Falls back to initials in orange circle.
<code>ui/skeleton.tsx</code>	Loading skeleton with shimmer animation. Used for all loading states.
<code>ui/tabs.tsx</code>	Wraps Radix Tabs. Orange active indicator underline. Smooth transition.
<code>ui/popover.tsx</code>	Wraps Radix Popover. Used for date pickers and filter panels.
<code>ui/dropdown-menu.tsx</code>	Wraps Radix DropdownMenu. Used in nav and action menus.
<code>ui/table.tsx</code>	Styled HTML table with sortable column headers. Striped rows. Orange sort indicators.
<code>ui/empty-state.tsx</code>	Centered illustration + heading + body + optional CTA button. Used when lists are empty.
<code>ui/spinner.tsx</code>	Orange circular spinner SVG. Sizes: sm, md, lg.

Step 3.3 [CODE] — Form wrappers (src/components/forms/)

File Path	Purpose / Contents
<code>forms/form-field.tsx</code>	Wraps react-hook-form Controller with label, error display, and helper text.
<code>forms/phone-input.tsx</code>	Phone number input with Somaliland prefix (+252) pre-filled. Validates E.164 format.
<code>forms/price-input.tsx</code>	SLSH amount input. Right-aligned number. Shows USD equivalent below using currency store rate.
<code>forms/image-upload.tsx</code>	Drag-and-drop + click to upload. Preview with remove button. Uploads to Supabase Storage via /api/admin/upload.
<code>forms/search-input.tsx</code>	Search bar with magnifying glass icon. Debounced onChange (300ms). Clear button when value present.
<code>forms/otp-input.tsx</code>	6-digit OTP entry with auto-advance between inputs. Paste support.

Step 3.4 [CODE] — Layout components (src/components/layout/)

File Path	Purpose / Contents
<code>layout/customer-header.tsx</code>	Sticky top nav: logo, search bar, language toggle, currency toggle, cart icon (badge count), user menu. White bg, shadow on scroll.
<code>layout/customer-footer.tsx</code>	Links: About, Contact, Privacy. Social links. Bilingual. Dark stone background.
<code>layout/customer-nav.tsx</code>	Mobile bottom nav: Home, Categories, Search, Orders, Account. Orange active indicator.
<code>layout/admin-sidebar.tsx</code>	Left sidebar with nav links per App Flow doc Admin Portal section. Collapsible. Active link = orange bg.
<code>layout/admin-header.tsx</code>	Top bar: current page title, notifications bell, user avatar + role badge, sign-out.
<code>layout/inventory-sidebar.tsx</code>	Same structure as admin sidebar but with inventory-specific links.
<code>layout/pos-layout.tsx</code>	Black background POS frame: top bar with session info, main content area, no standard nav.
<code>layout/staff-sidebar.tsx</code>	Simplified sidebar for staff portal.
<code>layout/portal-guard.tsx</code>	Client component that reads auth store role and renders children or redirects. Used in each portal layout.

Step 3.5 [CODE] — Shared feature components

File Path	Purpose / Contents
<code>components/product-card.tsx</code>	Product listing card: image, name (bilingual), price in SL\$H (USD below), stock badge, add-to-cart button.
<code>components/product-grid.tsx</code>	Responsive grid of ProductCards. Skeleton loading state.
<code>components/category-card.tsx</code>	Category icon + name. Used on home page and category listing.
<code>components/banner-carousel.tsx</code>	Auto-playing banner carousel with dot indicators. Uses CSS transitions (no external library).
<code>components/price-display.tsx</code>	Reads currency store. Shows SL\$H always; shows USD below in muted text when displayCurrency='USD'.
<code>components/loyalty-badge.tsx</code>	Tier badge (bronze/silver/gold) with tier-appropriate colour and Lucide icon.
<code>components/order-status-stepper.tsx</code>	Visual step tracker for order status flow.
<code>components/payment-method-icon.tsx</code>	Renders correct icon/logo for zaad/edahab/evc_plus/sahal/cash.
<code>components/language-toggle.tsx</code>	EN/SO toggle button. Updates next-intl locale and profile preference.
<code>components/currency-toggle.tsx</code>	SL\$H/USD toggle. Updates currency store.
<code>components/page-header.tsx</code>	Page title + optional subtitle + optional CTA button row. Used at top of every portal page.
<code>components/data-table.tsx</code>	Generic sortable/filterable/paginated table for admin lists.
<code>components/confirm-dialog.tsx</code>	Reusable destructive-action confirmation dialog. Title, message, confirm button (red).
<code>components/mobile-bottom-sheet.tsx</code>	Slide-up drawer for mobile. Wraps Radix Dialog with bottom positioning.

CHECKPOINT — verify before proceeding:

- ✓ All ui/ components render in isolation without errors
- ✓ All badge variants from Guidelines doc badge table are implemented and match spec
- ✓ Button variants all match exact colours/states from Guidelines doc
- ✓ CustomerHeader renders with correct sticky behaviour on mobile
- ✓ Skeleton shimmer animation is visible during loading states
- ✓ Language toggle updates all visible text on the page
- ✓ Currency toggle switches all PriceDisplay components between SLSH and USD

Phase 4 — Authentication

Goal: Implement complete phone OTP authentication flow as specified in Schema doc Section 2 and App Flow doc Section 2. Users authenticate with their Somaliland phone number via 6-digit SMS OTP. Role is loaded from the profiles table after successful verification.

Prerequisites (must be complete before this phase):

- Phase 2 complete — Supabase clients available
- Phase 3 complete — OTP input, phone input, button, form components available
- Phase 1 complete — Supabase Auth phone OTP configured

Step 4.1 [CODE] — API routes for auth

File Path	Purpose / Contents
<code>app/api/auth/send-otp/route.ts</code>	POST. Rate-limit: 5 requests per 5 min per IP. Calls <code>supabase.auth.signInWithOtp({ phone })</code> . Returns <code>{ success: true }</code> .
<code>app/api/auth/verify-otp/route.ts</code>	POST. Body: <code>{ phone, token }</code> . Calls <code>supabase.auth.verifyOtp()</code> . Returns session + profile. On first sign-in, profile is auto-created by trigger.
<code>app/api/auth/sign-out/route.ts</code>	POST. Calls <code>supabase.auth.signOut()</code> . Clears auth store. Redirects to <code>/sign-in</code> .
<code>app/api/auth/me/route.ts</code>	GET. Returns authenticated user's profile (<code>user_id</code> , <code>role</code> , <code>full_name</code> , <code>location_id</code> , <code>preferred_language</code> , <code>preferred_currency</code>).
<code>app/api/auth/profile/route.ts</code>	PATCH. Update <code>full_name</code> , <code>preferred_language</code> , <code>preferred_currency</code> . Rate-limit: 10/min.

Step 4.2 [CODE] — Auth pages

File Path	Purpose / Contents
<code>app/(auth)/sign-in/page.tsx</code>	Phone number entry screen. Orange brand header. Fields: phone number (auto-prefix +252). Submit calls <code>/api/auth/send-otp</code> . On success, navigate to <code>/verify?phone=XXX</code> . Show loading state on button. Show rate-limit error if hit.
<code>app/(auth)/verify/page.tsx</code>	OTP entry screen. Show last 4 digits of phone. <code>OtpInput</code> component (6 boxes). Resend OTP link (60s cooldown countdown). Submit calls <code>/api/auth/verify-otp</code> . On success: load profile, set auth store, redirect based on role (customer->/; admin->/admin/dashboard; etc.).

Step 4.3 [CODE] — Auth hook (`src/hooks/use-auth.ts`)

```
// src/hooks/use-auth.ts
// useAuth() returns: { user, profile, role, isLoading, signOut }
// - Reads from auth Zustand store
// - On mount: calls supabase.auth.getSession()
// - Subscribes to supabase.auth.onAuthStateChange()
// - On SIGNED_IN: fetches /api/auth/me and updates store
// - On SIGNED_OUT: clears store, clears cart store
```

```
// - Exports: useRequireAuth(requiredRole?) – redirects if not authenticated/authorized
```

Step 4.4 [CODE] — First-launch profile completion

New users have an empty `full_name` after OTP verification. Show a mandatory profile completion step before reaching home page:

```
// app/(auth)/complete-profile/page.tsx
// - Only reachable if profile.full_name is empty
// - Fields: Full Name (required), Language preference, Currency preference
// - On submit: PATCH /api/auth/profile
// - Redirect to intended destination stored in ?next= query param
```

CHECKPOINT — verify before proceeding:

- ✓ OTP SMS arrives on real device within 30 seconds
- ✓ Entering wrong OTP shows clear error message (not a 500)
- ✓ Correct OTP creates profile row in Supabase with role='customer'
- ✓ Admin user redirected to /admin/dashboard after sign-in
- ✓ Cashier redirected to /pos after sign-in
- ✓ Accessing /admin without auth redirects to /sign-in
- ✓ Sign-out clears session and redirects to /sign-in
- ✓ Profile completion screen appears on first sign-in for new users

Phase 5 — Customer Store: Catalog

Goal: Build all customer-facing product browsing pages — home page, category listing, product detail, and semantic search. All UI must match the Guidelines doc exactly. All copy must be bilingual (English + Somali). All pages must be PWA-friendly (fast, cacheable).

Prerequisites (must be complete before this phase):

- Phases 0-4 complete
- At least 5 test products with variants and images loaded in Supabase

Step 5.1 [CODE] — Public API routes for catalog

File Path	Purpose / Contents
api/products/route.ts	GET. Query params: category_id?, search?, location_id?, sort?, page?, limit?. Returns paginated products with first image and min variant price. Joins inventory for stock status.
api/products/[id]/route.ts	GET. Returns product with all variants, all images, inventory levels per location, and related products from same category.
api/categories/route.ts	GET. Returns full nested category tree as JSON (recursive CTE query in Supabase).
api/products/featured/route.ts	GET. Returns products where is_featured=true ordered by updated_at DESC. Limit 8.
api/public/banners/route.ts	GET. Returns active banners for current date + location filter. Sorted by sort_order.
api/public/exchange-rate/route.ts	GET. Returns latest exchange rate (most recent exchange_rates row). Cached with Next.js revalidate=300.

Step 5.2 [CODE] — Home page (app/(customer)/page.tsx)

The home page is a Server Component for initial HTML. Sections per App Flow doc Section 3:

File Path	Purpose / Contents
Section: Banner Carousel	Fetches /api/public/banners. Renders BannerCarousel component. Auto-plays every 5s. Respects prefers-reduced-motion.
Section: Category Grid	Fetches /api/categories (top-level only). Renders 2-column mobile / 4-column desktop grid of CategoryCard. Horizontal scroll on mobile.
Section: Featured Products	Fetches /api/products/featured. Renders ProductGrid with skeleton on loading.
Section: AI Project Estimator CTA	Full-width orange banner with Lucide Hammer icon. Links to /ai/estimate.
Section: Loyalty Teaser	If authenticated customer: show current points and tier. If not auth: show join prompt.

Step 5.3 [CODE] — Product listing (app/(customer)/products/page.tsx)

File Path	Purpose / Contents
URL params	?category=UUID&sort=price_asc price_desc newest name&page=1&search=query
Layout	Sidebar filters (desktop) / slide-up filter sheet (mobile). Main: ProductGrid. Pagination at bottom.
Filters	Category tree (collapsible), Price range (SLSH slider), In-stock only toggle, Brand checkboxes
Sort	Dropdown: Relevance, Price (Low to High), Price (High to Low), Newest, Name A-Z
Product count	Show '24 products found' above grid. Update when filters change.
Empty state	EmptyState component with 'No products found' + 'Clear filters' button when results are empty

Step 5.4 [CODE] — Product detail (app/(customer)/products/[id]/page.tsx)

Server Component for SEO. Per App Flow doc Section 4:

File Path	Purpose / Contents
Image gallery	Main image large. Thumbnails row. Click to swap main image. Zoom on desktop hover.
Variant selector	Dynamic attribute buttons (Size, Length, Finish, etc.) from variant.attributes JSONB. Selecting updates price and stock display.
Price display	Large SLSH price. USD equivalent below. Loyalty points to be earned shown ('Earn 125 points').
Stock status	In Stock (green), Low Stock (orange, show qty if < 5), Out of Stock (red disabled button).
Add to cart	Orange button. Quantity stepper (1..99). Disabled when out of stock. Success toast on add.
Product details	Tabs: Description (bilingual toggle), Specifications (from attributes), Brand.
Related products	Horizontal scroll row of ProductCards from same category. Max 6 items.
Breadcrumb	Home > Category > Sub-category > Product name. Accessible nav landmark.

Step 5.5 [CODE] — Search (app/(customer)/search/page.tsx)

File Path	Purpose / Contents
api/ai/search/route.ts	POST. Body: { query: string, location_id?: string }. 1) Embed query with Gemini text-embedding-004. 2) Call match_products_semantic() pgvector function. 3) Return ranked ProductWithVariants list. Rate-limit: 30/min.
Search page	SearchInput (large, auto-focused). As user types (debounced 400ms): show keyword results instantly from /api/products?search=. On submit: call /api/ai/search for semantic results. Show both keyword and semantic sections with clear labels.

File Path	Purpose / Contents
Search suggestions	Dropdown of recent searches from localStorage. 5 most recent. X to clear each.
No results	EmptyState with AI chatbot CTA: 'Not finding what you need? Ask our AI assistant'.

- CHECKPOINT — verify before proceeding:**
- ✓ Home page loads in < 2 seconds on slow 3G (Next.js Server Component + ISR)
 - ✓ BannerCarousel auto-plays and pauses on prefers-reduced-motion
 - ✓ Selecting a variant on product detail updates price without page reload
 - ✓ Search returns results within 3 seconds for semantic queries
 - ✓ All product names, descriptions, and category names display in both EN and SO
 - ✓ Products page filters work in combination (category + price + in-stock)
 - ✓ Breadcrumb on product detail is correct and accessible

Phase 6 — Customer Store: Cart, Orders & Loyalty

Goal: Complete the e-commerce transaction flow — cart management, checkout (mobile money + cash on pickup), order tracking with real-time status updates, loyalty points management, and return requests. All flows are specified in App Flow doc Sections 5-7.

Prerequisites (must be complete before this phase):

- Phases 0-5 complete — product catalog working
- Mobile money settings configured in Supabase for at least one provider

Step 6.1 [CODE] — Cart (src/stores/cart.store.ts + app/(customer)/cart/page.tsx)

File Path	Purpose / Contents
Cart store	CartItem: { variantId, productId, productNameEn, productNameSo, sku, image, priceSLSH, quantity }. Persisted in localStorage. Synced to idb for offline access.
Cart page	List of CartItems with image, name, variant attributes, unit price, qty stepper, remove button. Subtotal in SLSH (USD below). Checkout button (disabled if cart empty or any item out of stock).
Stock validation	On cart page load: call /api/cart/validate to check current stock for all items. Show warning badge on items that are now out of stock.
api/cart/validate	POST. Body: items[]. Returns: items with current available_qty. Flags items where requested qty > available.

Step 6.2 [CODE] — Checkout flow (app/(customer)/checkout/page.tsx)

Multi-step checkout as specified in App Flow doc Section 5.3. Steps:

File Path	Purpose / Contents
Step 1: Pickup Location	Select active branch from /api/locations. Show address, hours, prayer break if today applies.
Step 2: Payment Method	Radio cards for each enabled mobile money provider + Cash on Pickup. Show merchant number and instructions (bilingual) from mobile_money_settings.
Step 3: Discount / Loyalty	Optional discount code input (validate via /api/orders/validate-code). Optional loyalty redemption slider (if customer has points). Show updated total.
Step 4: Review	Order summary: items, subtotal, discount, loyalty discount, total. Payment method. Location. Accept and Place Order button.
api/orders/route.ts POST	Create order: validate stock, apply discount code, calculate loyalty, reserve inventory (trigger fires), snapshot exchange rate, generate pickup code, return order_id. Rate-limit: 10/min.
Step 5: Payment Instructions	After order created: show exact payment instructions for chosen provider. Display order number (BH-YYYY-NNNNN) and total. 'I have sent payment' button calls /api/orders/[id]/payment-sent.

Step 6.3 [CODE] — Order tracking (app/(customer)/orders/)

File Path	Purpose / Contents
app/(customer)/orders/page.tsx	List of customer's orders, newest first. Show: order number, date, status badge, total, pickup location. Filter by status.
app/(customer)/orders/[id]/page.tsx	Order detail: items list, payment status, pickup code (when ready_for_pickup), estimated timeline. Real-time status updates via Supabase Realtime subscription.
Real-time updates	Subscribe to orders table with filter id=eq.ORDER_ID. On status change: show toast ('Your order is ready for pickup!'), update status stepper without reload.
api/orders/[id]/cancel/route.ts	POST. Only allowed when status='pending_payment'. Updates order status, cancellation_reason. Trigger releases inventory reservation.

Step 6.4 [CODE] — Loyalty page (app/(customer)/loyalty/page.tsx)

File Path	Purpose / Contents
Card display	Large card with card number (BH-LC-XXXXXX), current tier badge, current points (big number), lifetime points, progress bar to next tier threshold.
Points history	Paginated table: date, transaction type (earn/redeem/adjust), points, source (order number or 'POS sale'). Newest first.
Redemption info	Current tier discount percentage. Minimum 100 points to redeem. Show how much SLSH value current points are worth at checkout.
api/loyalty/card	GET. Returns own loyalty_card with current_tier joined to loyalty_tiers for tier details.
api/loyalty/transactions	GET. Returns paginated loyalty_transactions for own card.

Step 6.5 [CODE] — Return requests (app/(customer)/returns/)

File Path	Purpose / Contents
app/(customer)/returns/new/page.tsx	Select source order. Select items to return. Enter reason for each item. Upload up to 3 photos per item via image-upload component. Submit calls /api/returns POST.
app/(customer)/returns/[id]/page.tsx	Return status, items, submitted photos, staff notes if rejected, refund details if approved.
api/returns/route.ts POST	Create return: validate order belongs to customer, validate items exist in order, upload photos to return-photos bucket, insert return + return_items. Rate-limit: 5/min.

CHECKPOINT — verify before proceeding:

- ✓ Cart persists items across browser refresh and page navigation
- ✓ Checkout creates an order row in Supabase with status='pending_payment'
- ✓ Inventory.quantity_reserved increments immediately on order creation (trigger)
- ✓ Discount code validation catches: expired, exceeded limit, minimum order not met
- ✓ Order with status='ready_for_pickup' shows pickup code on tracking page
- ✓ Real-time subscription updates order status without full page reload
- ✓ Cancelling a pending order releases the inventory reservation
- ✓ Return request creates rows in returns and return_items tables
- ✓ Loyalty points display correctly after a completed test order

Phase 7 — Admin Portal

Goal: Build the complete admin management portal including product CRUD, order management with payment confirmation, staff management, analytics dashboard, and all settings pages. All pages are behind role='admin' guard. Specifications in App Flow doc Sections 8-9.

Prerequisites (must be complete before this phase):

- Phases 0-6 complete
- At least one profile with role='admin' exists in Supabase

Step 7.1 [CODE] — Admin layout + navigation

File Path	Purpose / Contents
<code>app/(admin)/layout.tsx</code>	Import AdminSidebar, AdminHeader. Wrap in PortalGuard(requiredRole='admin'). Set bg-stone-50.
<code>app/(admin)/dashboard/page.tsx</code>	KPI cards: today's revenue (SLSH), orders today, low-stock alerts count, active POS sessions. Recent orders table (last 10). Revenue chart (last 7 days using CSS-only bar chart, no external chart library).
<code>components/admin/kpi-card.tsx</code>	Metric card: icon, label, value, trend vs yesterday (green up arrow / red down arrow).
<code>components/admin/revenue-chart.tsx</code>	Simple 7-day bar chart using divs + Tailwind. No recharts or chart.js — keep bundle small.

Step 7.2 [CODE] — Product management

File Path	Purpose / Contents
<code>app/(admin)/products/page.tsx</code>	DataTable of all products (incl. inactive). Columns: image thumbnail, name, SKU, category, variants count, is_featured toggle, is_active toggle, actions (edit, view). Searchable. Filterable by category and status.
<code>app/(admin)/products/new/page.tsx</code>	Multi-step product creation form: 1) Basic info (name EN/SO, description EN/SO, category, brand, unit, SKU base, tags). 2) First variant (SKU, price SLISH, cost price, attributes). 3) Images (multi-upload, drag to reorder). 4) Review and create. On submit: POST /api/admin/products, then trigger embedding via /api/ai/embed-product.
<code>app/(admin)/products/[id]/page.tsx</code>	Full product editor: all fields editable in-line. Variant list with add/edit/deactivate actions. Image gallery with reorder. AI regenerate description button.
<code>api/admin/products/route.ts</code>	GET: all products (admin view). POST: create product + first variant. Validates category exists.
<code>api/admin/products/[id]/route.ts</code>	PATCH: update product fields. Audit logged.
<code>api/admin/variants/[id]/route.ts</code>	PATCH: update variant. POST /api/admin/products/[id]/variants: add new variant.

File Path	Purpose / Contents
<code>api/admin/upload/route.ts</code>	POST. Receives image file. Sharp: resize to 1200px max, convert to WebP. Upload to product-images bucket. Create 300x300 thumbnail to product-thumbnails bucket. Return both URLs.

Step 7.3 [CODE] — Category management

File Path	Purpose / Contents
<code>app/(admin)/products/page.tsx (tab)</code>	Categories tab on product page (or separate /categories page). Nested tree view. Add/edit/deactivate categories. Icon upload.
<code>api/admin/categories/route.ts</code>	GET: full tree (admin, incl. inactive). POST: create category.
<code>api/admin/categories/[id]/route.ts</code>	PATCH: update category. Validate no circular parent reference.

Step 7.4 [CODE] — Order management

File Path	Purpose / Contents
<code>app/(admin)/orders/page.tsx</code>	Orders DataTable. Columns: order number, customer phone, location, status badge, payment method, total SLISH, date, actions. Filter by status, location, date range. Export to CSV button.
<code>Order detail drawer</code>	Slide-out panel on row click: full order details, items, payment timeline, action buttons.
<code>api/admin/orders/[id]/confirm-payment/route.ts</code>	PATCH. Validates staff has required role. Updates order status to payment_confirmed + payment_transactions. Triggers inventory decrement (via trigger). Sends SMS to customer.
<code>api/admin/orders/[id]/ready/route.ts</code>	PATCH. Updates status to ready_for_pickup. Generates pickup code if not already set. Sends 'Your order is ready' SMS.
<code>api/admin/orders/[id]/complete/route.ts</code>	PATCH. Updates status to completed. Awards loyalty points (trigger fires).
<code>api/admin/analytics/export/route.ts</code>	GET. Accepts ?from=&to=&location_id=. Returns CSV of orders. Uses PapaParse to generate CSV string.

Step 7.5 [CODE] — Staff management

File Path	Purpose / Contents
<code>app/(admin)/staff/page.tsx</code>	Staff list: avatar, name, role badge, location, phone, active/inactive toggle, edit button.
<code>Staff add flow</code>	Enter phone number -> look up if user exists. If yes: assign role + location. If no: 'User must sign in first with this phone number'. Never create users from admin — they self-register.
<code>api/admin/staff/route.ts</code>	GET: all profiles with role != 'customer'. POST: assign role to existing user (update profiles.role + location_id).
<code>api/admin/staff/[id]/route.ts</code>	PATCH: update role, location_id, is_active. Audit logged (Schema trigger on profiles).

Step 7.6 [CODE] — Settings pages

File Path	Purpose / Contents
<code>app/(admin)/settings/page.tsx</code>	Tabbed settings: General Locations Mobile Money Exchange Rate Banners Discounts
Settings: Exchange Rate	Current rate display. Form: new SLISH rate input. History table (last 10 rates with who set them and when).
Settings: Locations	List of branches. Add/edit location form. Per-location hours editor (7-day schedule with prayer break toggle). Mobile money providers per location.
Settings: Discounts	Discount code DataTable. Create/edit discount code form: code, type (% or fixed), value, dates, min order, max uses, scope.
Settings: Banners	Banner list with preview images. Create/edit: title EN/SO, image upload, CTA text EN/SO, CTA URL, dates, location scope, sort order.
<code>api/admin/exchange-rate/route.ts</code>	POST: insert new exchange_rates row. Returns new rate.
<code>api/admin/locations/route.ts</code>	GET all, POST create.
<code>api/admin/locations/[id]/route.ts</code>	PATCH update, plus nested /hours and /mobile-money sub-routes.
<code>api/admin/discounts/route.ts</code>	GET all, POST create.
<code>api/admin/discounts/[id]/route.ts</code>	PATCH update/deactivate.
<code>api/admin/banners/route.ts</code>	GET all, POST create.
<code>api/admin/banners/[id]/route.ts</code>	PATCH update/deactivate.

Step 7.7 [CODE] — Analytics dashboard

File Path	Purpose / Contents
<code>app/(admin)/analytics/page.tsx</code>	Date range picker (today/7d/30d/custom). Location filter. KPI cards: total revenue, order count, average order value, top-selling product. Tables: top 10 products by revenue, revenue by location, payment method breakdown.
<code>api/admin/analytics/route.ts</code>	GET. Accepts ?from=&to=&location_id=. Runs aggregate SQL queries via Supabase. Returns structured JSON. Cached for 5 minutes.

CHECKPOINT — verify before proceeding:

- ✓ Admin can create a product with 3 variants and 3 images end-to-end
- ✓ Product embedding is triggered after product creation (check product_embeddings table)
- ✓ Order payment confirmation sends SMS to customer phone
- ✓ Changing a user's role is reflected in audit_log table
- ✓ CSV export downloads correctly formatted file
- ✓ Exchange rate change is immediately reflected in customer-facing price displays
- ✓ Discount code validation correctly rejects expired and over-limit codes

Phase 8 — Inventory Portal

Goal: Build the complete inventory management interface for inventory managers. Stock reception, transfers, alerts, stocktake workflow, movement history, and AI-powered reorder forecasts. Behind role IN ('inventory_manager','admin') guard.

Prerequisites (must be complete before this phase):

→ Phases 0-7 complete

→ At least one profile with role='inventory_manager' exists

Step 8.1 [CODE] — Inventory dashboard (app/(inventory)/dashboard/page.tsx)

File Path	Purpose / Contents
Stock overview	Summary cards: total SKUs, total stock value (sum of qty * cost_price_slsh), low-stock count (red badge), out-of-stock count (red badge).
Alerts widget	Top 5 unresolved low-stock / out-of-stock alerts. Each with product name, current qty, threshold, quick-reorder button.
Recent movements	Table of last 20 stock_movements for own location. Type badge, product, qty change, performed by, date.
api/inventory/route.ts	GET. Returns inventory rows for auth user's location_id with product + variant details. Supports filter: low_stock_only, out_of_stock_only, search.

Step 8.2 [CODE] — Stock view (app/(inventory)/stock/page.tsx)

File Path	Purpose / Contents
Stock table	Columns: product image, name/SKU, category, quantity on hand, reserved, available, threshold, aisle/shelf, last restocked. Sortable. Filter by category, alert status. Search by name/SKU.
Inline edit	Click threshold or aisle_shelf to edit in-place. Auto-saves on blur via PATCH /api/inventory/[id].
Bulk import	Upload CSV (PapaParse) to update thresholds in bulk. Download template button.
api/inventory/[id]/route.ts	PATCH: update threshold, aisle_shelf. Audit logged.

Step 8.3 [CODE] — Receive stock (app/(inventory)/receive/page.tsx)

File Path	Purpose / Contents
Receive form	Search/barcode-scan product. Select variant. Enter quantity received and cost price. Optional notes. Submit button.
Barcode scan	Camera access via MediaDevices API. Scan SKU barcode using react-barcode for display; @zxing/library for reading. Fills product automatically.

File Path	Purpose / Contents
<code>api/inventory/receive/route.ts</code>	POST. Increments quantity_on_hand. Inserts stock_movements row (movement_type='receive'). Updates last_restocked_at. Resolves any low_stock alert for this product/location.
Batch receive	Option to add multiple products before submitting. Useful for deliveries.

Step 8.4 [CODE] — Stocktake workflow (app/(inventory)/stocktakes/)

File Path	Purpose / Contents
<code>app/(inventory)/stocktakes/page.tsx</code>	List of stocktakes for own location. Status badges: draft, submitted, approved, rejected. Start new stocktake button.
<code>app/(inventory)/stocktakes/[id]/page.tsx</code>	Stocktake sheet: table of all inventory items with system_quantity (read-only) and counted_quantity (editable). Discrepancy auto-calculated. Submit for approval button. Tabs: All Discrepancies only.
New stocktake flow	On create: snapshot current quantity_on_hand into stocktake_items.system_quantity for all products at location. Create stocktake row with status='draft'.
Approval (admin only)	When submitted: admin sees stocktake in admin portal. Approve or reject with reason. On approval: trigger fires to apply corrections and create stock_movements.
<code>api/inventory/stocktakes/route.ts</code>	GET: own location's stocktakes. POST: create new stocktake.
<code>api/inventory/stocktakes/[id]/route.ts</code>	PATCH: update items (counted_quantity), submit. Admin: approve/reject.

Step 8.5 [CODE] — Alerts and forecasts

File Path	Purpose / Contents
<code>app/(inventory)/alerts/page.tsx</code>	List of inventory_alerts for own location. Low stock (orange) and out of stock (red) badges. Resolve button. Link to receive stock.
<code>app/(inventory)/forecasts/page.tsx</code>	AI reorder forecasts table: product, predicted stockout date, recommended reorder qty, confidence score, Gemini reasoning text. Refresh button triggers /api/ai/forecast.
<code>api/inventory/alerts/route.ts</code>	GET: unresolved alerts. PATCH /[id]: mark as resolved.
<code>api/inventory/forecasts/route.ts</code>	GET: latest ai_forecasts for own location. POST: trigger Gemini forecast generation for a product (builds prompt with 90-day sales history, calls gemini-1.5-pro, parses response, upserts ai_forecasts).

CHECKPOINT — verify before proceeding:

- ✓ Receiving 10 units of a product increments quantity_on_hand and creates a stock_movement row
- ✓ Creating a stocktake snapshots all current quantities correctly
- ✓ Approving a stocktake with discrepancies corrects inventory and creates stock_movements
- ✓ Low-stock alert auto-resolves when stock is received above threshold
- ✓ AI forecast returns structured JSON with predicted_stockout_date and recommended_reorder_qty
- ✓ Barcode scan fills product field on receive page

Phase 9 — POS Terminal

Goal: Build the complete in-store POS terminal as specified in App Flow doc Section 10. The POS must work offline with transaction queuing and background sync. It supports split payments, customer lookup, loyalty point award, receipt generation, transaction void, and parked cart management. Black background UI.

Prerequisites (must be complete before this phase):

- Phases 0-8 complete
- At least one profile with role='cashier' exists
- Phase 12 (PWA/Service Worker) is best done BEFORE this phase if offline support is required at launch

Step 9.1 [CODE] — Session management (app/(pos)/session/page.tsx)

File Path	Purpose / Contents
Open session form	Cashier selects location (if location_id not set on profile), enters starting cash float (SLSH + USD fields). Submit calls /api/pos/sessions POST.
Session status bar	Visible at top of POS: cashier name, location, session open time, total sales so far. Always visible during session.
Close session form	Enter ending cash count (SLSH + USD). Show calculated variance. Confirm close. Calls /api/pos/sessions/[id]/close PATCH.
api/pos/sessions/route.ts	POST: create session (checks no open session exists for cashier). GET: return active session for current cashier.
api/pos/sessions/[id]/route.ts	PATCH: close session with ending cash. Calculate cash_variance.
api/pos/sessions/active/route.ts	GET: returns currently open session for auth user. Returns null if none. Called on POS mount.

Step 9.2 [CODE] — POS main screen (app/(pos)/page.tsx)

File Path	Purpose / Contents
Layout	Black background full-screen. Left panel: product search + cart. Right panel: payment + session totals. No standard navigation.
Product search bar	Large search input (white text on dark bg). Search by name, SKU, or barcode. Results appear in dropdown list. Click to add to cart.
Barcode scanner	Camera button activates barcode scanner. On scan: lookup /api/pos/products/barcode/[sku] and add to cart.
Cart panel	List of cart items: name, qty stepper, unit price, line total. All in SLSH. Remove button per item. Cart total at bottom.
Customer panel	Optional: phone number input. Lookup button calls /api/pos/customer-lookup. Shows: customer name, loyalty card, points balance, tier badge. 'Link to sale' checkbox to award points.
Discount code	Code input field. Apply button. Shows discount amount if valid. Validates via /api/orders/validate-code (reused endpoint).

File Path	Purpose / Contents
Parked carts	Parked cart drawer/panel. List of parked carts with items preview and park time. Recall button restores cart.
Function buttons	Park Cart, Clear Cart, Void Last, New Sale. All accessible with keyboard shortcuts (F1-F4).

Step 9.3 [CODE] — Payment flow

File Path	Purpose / Contents
Charge button	Large green button: 'Charge TOTAL_SLASH'. Opens payment modal.
Payment modal	Select payment method(s). For cash: enter amount tendered, show change due. For mobile money: enter transaction reference. Support split: add multiple payment methods (total must equal cart total).
Cash payment	amount_tendered input. change_slash = tendered - total. Confirm button creates transaction immediately.
Mobile money payment	Reference field. 'Awaiting confirmation' state. Staff verifies reference separately and marks confirmed via Staff Portal.
api/pos/transactions/route.ts	POST: creates pos_transaction + items + payment_splits. Decrements inventory (direct UPDATE, no reservation needed for POS). Awards loyalty points if customer linked (trigger fires). Inserts stock_movements. Returns transaction with receipt data.
Receipt generation	After successful transaction: generate PDF receipt using jsPDF. Includes: store name/location, transaction number, date, cashier name, items table, subtotal, discount, total, payment method, change given, loyalty points earned, QR code linking to receipt. Print via window.print() or offer PDF download.

Step 9.4 [CODE] — Void, park/recall, and offline sync

File Path	Purpose / Contents
Transaction void	Void button on completed transaction. Requires void_reason. Manager PIN or dual-auth if configured. Calls /api/pos/transactions/[id]/void POST. Reverses inventory (stock_movement type='void'). Voids loyalty points earned.
Park cart	Park button saves current cart_data JSONB to parked_transactions via /api/pos/park. Cart cleared for new sale. Parked cart expires in 4 hours (pg_cron).
Recall cart	Parked cart list shows all active parked carts for this session. Recall: load items back into cart. Calls /api/pos/parked/[id]/recall PATCH.
Offline mode	Service worker (Phase 12) intercepts POS API calls when offline. Transactions queued in idb (IndexedDB). Status bar shows 'OFFLINE MODE — X transactions queued'. On reconnect: Background Sync API triggers /api/pos/sync POST to batch-submit queued transactions.
api/pos/sync/route.ts	POST. Accepts array of offline transactions. Processes each sequentially. Returns success/failure per transaction. Failed transactions (e.g. stock depleted) flagged for cashier review.

Step 9.5 [CODE] — Online order pickup and payment verification

File Path	Purpose / Contents
<code>api/pos/pickup/verify/route.ts</code>	POST. Body: { pickup_code }. Finds order by pickup_code at cashier's location. Returns order details. Marks order as completed on confirm. Awards loyalty points.
<code>api/pos/payment/confirm/route.ts</code>	POST. Body: { order_id, transaction_reference }. Confirms mobile money payment for online order. Updates payment_transactions status='confirmed'. Triggers order status -> payment_confirmed.
Pickup lookup in POS	Tab in POS: 'Pickup'. Enter 6-char pickup code. Show order details + items. Confirm pickup button. Prints confirmation receipt.

CHECKPOINT — verify before proceeding:

- ✓ Opening a POS session creates a pos_sessions row with status='open'
- ✓ Adding items and completing a cash sale creates pos_transaction, items, and payment_split rows
- ✓ Inventory quantity_on_hand decrements immediately on POS sale
- ✓ Loyalty points are awarded when customer phone is linked to sale
- ✓ Parked cart appears in parked list and can be recalled
- ✓ Going offline (disable network in DevTools) queues transactions in IndexedDB
- ✓ Re-enabling network triggers background sync and transactions appear in Supabase
- ✓ Pickup code lookup finds the correct online order
- ✓ Receipt PDF downloads correctly with all transaction details

Phase 10 — Staff Portal

Goal: Build the lightweight staff portal for managing online orders — confirming mobile money payments, processing pickups, and handling return assessments. Accessible to cashier, inventory_manager, and admin roles.

Prerequisites (must be complete before this phase):

- Phases 0-9 complete
- Online orders exist in the system for testing

Step 10.1 [CODE] — Staff dashboard (app/(staff)/layout.tsx + overview)

File Path	Purpose / Contents
app/(staff)/layout.tsx	StaffSidebar + PortalGuard(isStaff=true). Shows location name in header.
Staff overview	Cards: pending payment confirmations count, orders ready for pickup count, pending return requests count. Quick-action buttons to each section.

Step 10.2 [CODE] — Payment confirmation (app/(staff)/payments/page.tsx)

File Path	Purpose / Contents
Pending payments list	Orders with status='payment_submitted' at staff's location. Columns: order number, customer phone, payment method, amount SLISH, reference number, submitted time.
Confirm action	Click order -> detail drawer. Shows customer-submitted reference. Staff verifies in their mobile money app independently. 'Confirm Payment' button -> calls /api/admin/orders/[id]/confirm-payment. 'Report Issue' -> marks payment_status='failed' with note.
Real-time updates	Supabase Realtime subscription on orders table WHERE location_id=staff_location AND status='payment_submitted'. New pending payments appear in real time with notification sound.

Step 10.3 [CODE] — Pickup management (app/(staff)/pickup/page.tsx)

File Path	Purpose / Contents
Ready for pickup list	Orders with status='ready_for_pickup' at staff's location. Sorted by oldest first.
Pickup verification	Large pickup code input field (or barcode scan). On submit: calls /api/pos/pickup/verify. Shows order items to physically verify. 'Confirm Handover' marks order complete.
Walk-in pickup	Button to lookup any order by order number or customer phone (fallback for lost pickup code).

Step 10.4 [CODE] — Return processing (app/(staff)/returns/page.tsx)

File Path	Purpose / Contents
Pending returns list	Returns with status='pending' at staff's location. Customer name, items count, submitted date.
Return detail	View submitted photos, item reasons. Staff inputs: approve/reject decision, refund method (original/cash/store credit), refund amount SLISH, mobile money phone for refund, rejection reason if rejected.
Approve action	PATCH /api/returns/[id] with status='approved'. If mobile money refund: admin processes manually (no automated payout). Staff records refund_reference once sent.

CHECKPOINT — verify before proceeding:

- ✓ Confirming a payment updates order status to 'payment_confirmed' and sends SMS to customer
- ✓ Order moves to 'ready_for_pickup' status after staff marks it ready
- ✓ Pickup verification marks order as 'completed'
- ✓ New pending payments appear in real time without page refresh
- ✓ Approving a return creates a stock movement (if items returned to inventory)

Phase 11 — AI Features

Goal: Integrate all AI-powered features using the Google Gemini API. Chatbot assistant, project material estimator, semantic product search, AI product description generation, and inventory forecasting. All AI calls are rate-limited and error-handled gracefully.

Prerequisites (must be complete before this phase):

- Phases 0-10 complete
- GEMINI_API_KEY set in .env.local
- At least 20 products with embeddings in product_embeddings table (needed for semantic search quality)

Step 11.1 [CODE] — AI Hardware Chatbot

File Path	Purpose / Contents
api/ai/chat/route.ts	POST. Body: { messages: ChatMessage[], language: 'en' 'so' }. Rate-limit: 5/min per IP. Uses gemini-1.5-flash streaming. System prompt: 'You are a hardware store assistant for Borama Hardware in Somaliland. Help customers find products, explain building materials, and answer hardware questions. Respond in {language}. Keep responses concise and practical.' Streams response using Next.js streaming.
components/ai/chat-widget.tsx	Floating chat button (bottom-right, orange). Opens slide-in panel. Messages list: customer bubbles (orange), AI bubbles (white). Input bar + send button. Typing indicator (3 dot animation). Renders markdown in AI responses. Language follows app locale.
Chat state	Kept in local component state (not persisted — chat history is session-only). Max 20 messages before suggesting 'Start new conversation'.

Step 11.2 [CODE] — Project Material Estimator

File Path	Purpose / Contents
api/ai/estimate/route.ts	POST. Body: { description: string, area_sqm?: number, language: 'en' 'so' }. Rate-limit: 3/min per IP. Uses gemini-1.5-pro (not flash — needs accuracy). Prompt instructs Gemini to: 1) Identify the project type. 2) List required materials with quantities and Somali unit prices (in SL\$). 3) Return structured JSON: { project_type, materials: [{name, quantity, unit, unit_price_slsh, total_slsh}], total_slsh, notes }. Parse JSON response. Look up actual prices from products table. Return combined result.
app/(customer)/ai/estimate/page.tsx	Project Estimator page. Form: project description textarea (e.g. 'I want to build a 3-room house, 80 square meters'). Area input (optional). Language toggle. Estimate button. Results: animated reveal of materials table with SL\$ prices and USD equivalents. Total. 'Add all to cart' button. Print/download as PDF button.

Step 11.3 [CODE] — Semantic product search embedding pipeline

File Path	Purpose / Contents
api/ai/embed-product/route.ts	POST. Body: { product_id, language?: 'en' 'so' 'combined' }. Admin/inventory_manager only. Builds embedding text: '[name_en] [name_so] [description_en] [description_so] [tags] [category] [brand] [variant attributes]'. Calls Gemini text-embedding-004 to get 768-dimension vector. Upserts product_embeddings row. Returns success.
Batch embedding script	src/scripts/batch-embed.ts — run via 'pnpm tsx src/scripts/batch-embed.ts'. Loops through all active products without embeddings and calls embed-product API. Adds 500ms delay between calls to respect rate limits. Shows progress bar.
api/ai/search/route.ts	POST. Body: { query, location_id? }. Embed query with text-embedding-004. Call match_products_semantic() Supabase RPC. Fetch full product details for returned product_ids. Return ranked list. Rate-limit: 30/min.

Step 11.4 [CODE] — AI product description generator

File Path	Purpose / Contents
api/ai/generate-description/route.ts	POST. Admin only. Body: { product_name_en, product_name_so, category, brand, attributes }. Prompt: 'Write a compelling product description for a hardware store in Borama, Somaliland. Product: {name}. Category: {category}. Brand: {brand}. Attributes: {attributes}. Return JSON: { description_en: string, description_so: string }. Keep each description 2-3 sentences. Focus on practical use cases for construction in Somaliland.' Returns parsed JSON.
Usage in admin product form	'Generate with AI' button next to description fields. Shows spinner while generating. Populates both description_en and description_so fields. Staff can edit before saving.

Step 11.5 [CODE] — Inventory forecasting

File Path	Purpose / Contents
api/inventory/forecasts/route.ts POST	Trigger forecast for a product/location. Build prompt: sales velocity (last 90 days of stock_movements WHERE movement_type='sale'), current quantity_on_hand, threshold. Call gemini-1.5-pro. Parse JSON response: { predicted_stockout_date, recommended_reorder_qty, confidence_score (0-1), reasoning_text }. Upsert ai_forecasts row.
Weekly auto-forecast	pg_cron job (weekly, Sunday midnight): trigger forecasts for all products that have been sold in last 30 days. Run via Supabase Edge Function or pg_cron calling a webhook.

CHECKPOINT — verify before proceeding:

- ✓ Chatbot responds in correct language (en/so) within 5 seconds
- ✓ Material estimator returns structured JSON and displays materials table
- ✓ Running batch-embed.ts populates product_embeddings for all products
- ✓ Semantic search returns relevant results (test: search 'cement for foundations')
- ✓ AI description generation fills both EN and SO fields correctly
- ✓ Inventory forecast inserts a row into ai_forecasts table with confidence score
- ✓ All AI endpoints return 429 when rate limit exceeded (not 500)

Phase 12 — PWA & Offline Mode

Goal: Configure the application as a fully installable Progressive Web App. Offline support is critical for the POS terminal — cashiers must be able to complete transactions even when internet connectivity is unstable. Uses next-pwa 5.6.0 with Workbox under the hood.

Prerequisites (must be complete before this phase):

- Phases 0-11 complete
- next-pwa 5.6.0 installed
- next.config.js must be CommonJS format (.js not .ts) — already enforced in Phase 0

Step 12.1 [CODE] — PWA manifest and icons

File Path	Purpose / Contents
public/manifest.json	name: 'Borama Hardware', short_name: 'Borama HW', start_url: '/', display: 'standalone', background_color: '#FAFAF9', theme_color: '#F97316', icons: [72, 96, 128, 144, 152, 192, 384, 512]px PNGs in public/icons/
public/icons/	Generate all required icon sizes from the store logo. Use sharp CLI or online tool. Must include maskable version (192x192 and 512x512 with safe zone).
app/layout.tsx	Add <link rel='manifest' href='/manifest.json' /> in metadata. Add apple-touch-icon meta. Set theme-color meta tag.

Step 12.2 [CODE] — Service worker configuration (next.config.js)

```
// next.config.js (CommonJS – DO NOT convert to .ts or .mjs)
const withPWA = require('next-pwa')({
  dest: 'public',
  register: true,
  skipWaiting: true,
  disable: process.env.NODE_ENV === 'development',
  runtimeCaching: [
    {
      // Cache API routes for product catalog (stale-while-revalidate)
      urlPattern: /^\/api\/(products|categories|public)/,
      handler: 'StaleWhileRevalidate',
      options: { cacheName: 'api-catalog', expiration: { maxAgeSeconds: 300 } },
    },
    {
      // Cache Supabase Storage images (cache-first, 7 days)
      urlPattern: /\.supabase\.co\/storage/,
      handler: 'CacheFirst',
      options: { cacheName: 'supabase-images', expiration: { maxAgeSeconds: 604800 } },
    },
    {
      // POS API routes – NetworkFirst (try network, fallback to queue)
      urlPattern: /^\/api\/pos/,
      handler: 'NetworkOnly', // Handled by custom offline queue instead
    }
  ]
})
```

```

},
],
});
module.exports = withPWA({ /* rest of next config */ });

```

Step 12.3 [CODE] — Offline POS transaction queue (idb)

File Path	Purpose / Contents
src/lib/offline-queue.ts	Initialize IndexedDB (idb 8.0.0). Store name: 'pos-offline-queue'. Schema: { id: auto, transaction_data: object, queued_at: ISO string, status: 'pending' 'synced' 'failed' }.
Queue write	In POS transaction flow: if navigator.onLine === false, write to idb instead of calling API. Show 'OFFLINE — transaction queued' toast.
Queue read + sync	On 'online' event and on Background Sync 'pos-sync' event: read all pending items from idb, call /api/pos/sync, mark each as synced or failed.
Background Sync registration	In service worker: register for Background Sync tag 'pos-sync'. This ensures sync happens even if user closes the tab (PWA background capability).
Sync status display	POS status bar shows: ONLINE (green dot) / OFFLINE (red dot) / SYNCING (orange spinner). When offline: shows queued transaction count.

Step 12.4 [CODE] — App install prompt

File Path	Purpose / Contents
components/pwa-install-prompt.tsx	Listen for 'beforeinstallprompt' event. Show branded install banner at bottom of screen after user has visited 2 times (tracked in localStorage). 'Add to Home Screen' button triggers prompt. Dismiss stores flag to not show again for 7 days.
Offline fallback page	app/offline/page.tsx — shown by service worker when user is offline and requests a non-cached page. Shows: Borama Hardware logo, 'You are offline' message, 'The POS terminal still works offline' note, retry button.

CHECKPOINT — verify before proceeding:

- ✓ Lighthouse PWA score >= 90 in Chrome DevTools
- ✓ App installs on Android Chrome via 'Add to Home Screen'
- ✓ App installs on iOS Safari (add to home screen manually)
- ✓ Completed POS transaction while offline appears in idb queue
- ✓ Going online syncs queued transactions to Supabase
- ✓ Product images load from cache when offline (previously visited products)
- ✓ Offline fallback page displays when navigating to uncached route offline
- ✓ 'Add to Home Screen' prompt appears on second visit to customer store

Phase 13 — Testing

Goal: Write a comprehensive test suite covering business-critical paths. Unit tests with Vitest for utilities and pure functions. Component tests for key UI components. End-to-end tests with Playwright for the most critical user journeys. Target: all critical paths have at least one E2E test before deployment.

Prerequisites (must be complete before this phase):

- All feature phases complete (Phases 0-12)
- A dedicated test Supabase project OR seeded test data that can be reset between runs

Step 13.1 [CODE] — Unit tests (Vitest)

File Path	Purpose / Contents
<code>src/lib/__tests__/utils.test.ts</code>	Test: <code>cn()</code> merges classes correctly. <code>formatSLSH(125000) = '125,000 SLSH'</code> . <code>slshToUsd(570000, 570) = '\$1000.00'</code> . <code>truncate()</code> respects max length. <code>getInitials()</code> handles edge cases.
<code>src/lib/__tests__/ratelimit.test.ts</code>	Mock Upstash Redis. Verify rate limiter allows N requests and blocks N+1.
<code>src/stores/__tests__/cart.test.ts</code>	Test cart store: <code>addItem</code> , <code>removeItem</code> , <code>updateQty</code> , <code>applyCode</code> , <code>clearCart</code> . Verify totals calculate correctly with and without discount codes.
<code>src/stores/__tests__/currency.test.ts</code>	Test currency toggle switches display currency. Test <code>formatSLSH</code> and <code>slshToUsd</code> with mocked exchange rate.
<code>src/lib/__tests__/env.test.ts</code>	Verify <code>env.ts</code> throws when required vars missing. Verify it parses correctly with all vars set.
Run command	<code>pnpm vitest run</code> (or <code>pnpm vitest</code> for watch mode)

Step 13.2 [CODE] — Component tests (Vitest + Testing Library)

File Path	Purpose / Contents
<code>ui/__tests__/button.test.tsx</code>	Renders all variants. Disabled state prevents click. Loading state shows spinner. Calls <code>onClick</code> handler.
<code>ui/__tests__/badge.test.tsx</code>	All 18 badge variants render correct text and colours per Guidelines doc.
<code>components/__tests__/product-card.test.tsx</code>	Renders product name, price in SLASH, USD equivalent, stock badge. Out-of-stock disables add-to-cart button.
<code>components/__tests__/otp-input.test.tsx</code>	Typing in first box moves focus to second. Paste of 6 digits fills all boxes. Backspace moves to previous box.
<code>components/__tests__/price-display.test.tsx</code>	Shows SLASH when <code>currency='SLASH'</code> . Shows USD when <code>currency='USD'</code> . Updates when exchange rate changes.

Step 13.3 [CODE] — API route tests

File Path	Purpose / Contents
<code>api/__tests__/orders.test.ts</code>	Mock Supabase admin client. POST /api/orders: validates cart items in stock, creates order, returns order_number. Test: insufficient stock returns 409. Test: invalid discount code returns 400.
<code>api/__tests__/discount.test.ts</code>	Test check_discount_code_validity function directly: expired code returns false. Over-limit returns false. Valid code returns correct discount_amount.
<code>api/__tests__/ai-search.test.ts</code>	Mock Gemini embedText function. Mock Supabase RPC call. Verify search returns ranked results.
<code>api/__tests__/pos-transaction.test.ts</code>	Mock Supabase admin. POST /api/pos/transactions: verifies stock decremented, loyalty awarded, movement created.

Step 13.4 [CODE] — End-to-end tests (Playwright)

File Path	Purpose / Contents
<code>e2e/auth.spec.ts</code>	Test full OTP auth flow: enter phone, receive OTP (use Twilio test credentials), enter OTP, verify redirect to home, sign out.
<code>e2e/customer-checkout.spec.ts</code>	Browse products, add to cart, apply discount code, select pickup location, select payment method, place order, verify order appears in orders list.
<code>e2e/admin-product.spec.ts</code>	Sign in as admin, create product with 2 variants, verify appears in product listing, edit product name, verify change saved.
<code>e2e/pos-transaction.spec.ts</code>	Sign in as cashier, open session, search product by SKU, add to cart, process cash payment, verify transaction in pos_transactions table.
<code>e2e/loyalty.spec.ts</code>	Complete an order as customer, navigate to loyalty page, verify points earned match order total / 1000.
<code>playwright.config.ts</code>	Base URL from PLAYWRIGHT_BASE_URL env. 3 browsers: chromium, firefox, webkit. Retry: 2 on CI. Screenshot on failure. Video on first retry.
Run command	<code>pnpm playwright test</code> (requires: <code>pnpm playwright install</code>)

CHECKPOINT — verify before proceeding:

- ✓ `pnpm vitest run` exits with 0 (all unit tests pass)
- ✓ `pnpm playwright test chromium` exits with 0 (all E2E tests pass)
- ✓ Code coverage report shows > 70% coverage on `src/lib/`
- ✓ No TypeScript errors: `pnpm tsc --noEmit` exits with 0
- ✓ No ESLint errors: `pnpm eslint src/` exits with 0
- ✓ No Prettier violations: `pnpm prettier --check src/` exits with 0

Phase 14 — Deployment

Goal: Deploy the application to Vercel, configure all production environment variables, connect a custom domain, run a performance and security audit, and execute the go-live checklist before handing to the Borama Hardware team.

Prerequisites (must be complete before this phase):

- Phase 13 complete — all tests pass
- Production Supabase project created (separate from dev)
- Domain name ready (e.g. borama-hardware.so)
- Vercel account connected to GitHub repo

Step 14.1 [CLI] — Production Supabase setup

File Path	Purpose / Contents
New Supabase project	Create a separate production Supabase project (different from development). Region: closest to Somaliland (Mumbai or Bahrain).
Run all migrations	Run all SQL from Phase 1 steps 1.1-1.9 in the production project. Verify all 35 tables, functions, triggers, and RLS policies created.
Seed production data	Insert loyalty_tiers (bronze/silver/gold). Insert default location(s). Set initial exchange rate. Configure mobile money providers.
Auth configuration	Configure Twilio credentials in production Supabase Auth settings. Test OTP delivery to a real Somaliland number. Register JWT hook.
Storage buckets	Create all 6 storage buckets with correct public/private settings.

Step 14.2 [CLI] — Vercel project setup

```
# Install Vercel CLI
pnpm add -g vercel

# Link project to Vercel
vercel link

# Set all production environment variables in Vercel dashboard:
# Project Settings > Environment Variables > Production
# Add all variables from Schema doc Section 15.2
# Use PRODUCTION Supabase URL and keys (NOT development)

# Deploy to production
vercel --prod

# Or: push to main branch if auto-deploy is configured
git push origin main
```

Step 14.3 [VERIFY] — Environment variable checklist

File Path	Purpose / Contents
NEXT_PUBLIC_SUPABASE_URL	Must point to PRODUCTION Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Production anon key (safe to expose — RLS enforced)

File Path	Purpose / Contents
SUPABASE_SERVICE_ROLE_KEY	Production service role key — verify NOT in NEXT_PUBLIC_ prefix
GEMINI_API_KEY	Google AI Studio API key — verify quota is sufficient
UPSTASH_REDIS_REST_URL	Production Upstash Redis instance URL
UPSTASH_REDIS_REST_TOKEN	Production Upstash token
NEXT_PUBLIC_APP_URL	Must be the production domain, e.g. https://borama-hardware.so
NEXT_PUBLIC_DEFAULT_LOCATION_ID	UUID of the main production location from locations table

Step 14.4 [VERIFY] — Domain and HTTPS

File Path	Purpose / Contents
Custom domain	Add domain in Vercel Project Settings > Domains. Add CNAME record in DNS provider: www -> cname.vercel-dns.com. Add A record for apex domain.
HTTPS	Vercel auto-provisions SSL certificate via Let's Encrypt. Verify at https://your-domain.so in browser.
PWA on domain	Verify manifest.json is accessible at https://your-domain.so/manifest.json. Verify service worker registers at scope '/' on production domain.

Step 14.5 [VERIFY] — Performance audit

File Path	Purpose / Contents
Lighthouse	Run Lighthouse on production URL in Chrome DevTools. Targets: Performance >= 85, Accessibility >= 95, Best Practices >= 95, SEO >= 90, PWA >= 90.
Core Web Vitals	LCP < 2.5s. FID/INP < 100ms. CLS < 0.1. Check in Chrome DevTools > Performance tab.
Image optimization	Verify all product images are served as WebP. Check Network tab. Images should load < 200KB per product detail page.
Bundle size	Run: pnpm build and check .next/analyze output. Total JS bundle < 300KB gzipped for customer-facing pages.

Step 14.6 [VERIFY] — Security audit

File Path	Purpose / Contents
RLS verification	In Supabase: run SELECT * FROM audit_log using the anon key (via Supabase JS client with no session). Must return 0 rows (denied by RLS).
Service role exposure	Search entire codebase for 'SUPABASE_SERVICE_ROLE_KEY' in any file under src/app/(customer), src/app/(auth), src/components. Must be 0 results.
API rate limits	Test each rate-limited endpoint beyond its limit. Must return HTTP 429 with appropriate message.

File Path	Purpose / Contents
Environment security	Verify no .env*.local files are committed: git ls-files .env* must return empty.
CSP headers	Add Content-Security-Policy header in next.config.js. Minimum: default-src 'self'; script-src 'self'; connect-src 'self' *.supabase.co *.googleapis.com.

Step 14.7 [VERIFY] — Go-live checklist

Final verification before opening to real customers:

1. Send real OTP to Somaliland phone number — receives within 30 seconds
2. Create test product as admin — appears in customer store
3. Place test order as customer — order appears in admin panel
4. Confirm payment as staff — customer receives 'payment confirmed' SMS
5. Mark ready for pickup — customer receives 'ready for pickup' SMS with pickup code
6. Complete pickup via POS pickup verification
7. Verify loyalty points awarded to test customer
8. Open POS session, process cash sale, close session
9. Go offline on POS device, process transaction, go online — verify sync
10. Install as PWA on Android device — launches as standalone app
11. Load home page on slow 3G simulation — LCP < 3 seconds
12. Switch language to Somali — all visible text changes
13. Switch currency to USD — all prices update correctly
14. Admin: change exchange rate — customer-facing USD prices update immediately
15. Test return flow: submit return as customer, approve as staff

CHECKPOINT — verify before proceeding:

- ✓ All 15 go-live checklist items pass on production domain
- ✓ Lighthouse scores meet targets on production URL
- ✓ No console errors in browser on any page (customer, admin, POS)
- ✓ Sentry or Vercel logging shows no unexpected 500 errors during go-live test
- ✓ At least one admin user, one cashier, and one inventory_manager profile exist in production
- ✓ Exchange rate is set to current USD/SLSH rate in production
- ✓ All active branches are configured with hours and mobile money settings
- ✓ Product catalog has been imported (admin created or CSV bulk import)
- ✓ All product embeddings are generated (batch-embed.ts run on production)
- ✓ DNS propagation complete — domain accessible from Somaliland ISPs

Build Order Reference

Quick reference card showing the exact sequence and what each phase produces. Use this to communicate status to stakeholders.

Phase	Name	Must Complete Before	Key Deliverable
0	Project Initialization	—	Next.js repo with all deps, configs, directory structure
1	Supabase Database	Phase 0	All 35 tables, functions, triggers, RLS policies, seed data
2	Core Infrastructure	Phases 0-1	Supabase clients, types, i18n, Zustand stores, middleware
3	Global UI System	Phase 2	All reusable components matching Guidelines doc exactly
4	Authentication	Phases 2-3	Phone OTP flow, protected routes, profile creation
5	Customer Catalog	Phases 3-4	Home page, product listing, product detail, search
6	Customer Orders	Phases 4-5	Cart, checkout, order tracking, loyalty, returns
7	Admin Portal	Phase 6	Product/order/staff/analytics/settings management
8	Inventory Portal	Phase 7	Stock management, stocktakes, alerts, forecasts
9	POS Terminal	Phases 7-8	Full POS with split pay, void, park, offline queue
10	Staff Portal	Phase 9	Payment confirmation, pickup, return processing
11	AI Features	Phases 5, 7, 8	Chatbot, estimator, semantic search, descriptions, forecasts
12	PWA & Offline	Phase 9	Service worker, offline POS queue, background sync, install
13	Testing	Phases 0-12	Vitest unit tests, Playwright E2E, all critical paths green
14	Deployment	Phase 13	Live on production domain, verified against go-live checklist

Estimated Total Build Time

The estimates below assume Claude Code builds continuously with no interruptions and the input documents (PRD, App Flow, Tech Stack, Guidelines, Schema) are referenced throughout.

Phase 0 (Init) ~1 hour
Phase 1 (Database) ~2 hours
Phase 2 (Foundation) ~2 hours
Phase 3 (UI System) ~4 hours
Phase 4 (Auth) ~2 hours
Phase 5 (Catalog) ~4 hours
Phase 6 (Orders) ~4 hours
Phase 7 (Admin) ~5 hours
Phase 8 (Inventory) ~3 hours

```
Phase 9 (POS) ~5 hours
Phase 10 (Staff) ~2 hours
Phase 11 (AI) ~3 hours
Phase 12 (PWA/Offline) ~2 hours
Phase 13 (Testing) ~3 hours
Phase 14 (Deploy) ~1 hour
```

Total: ~43 hours of Claude Code sessions

How to resume after context limit:

When a Claude Code session hits its context limit mid-phase, begin the next session with:

'Continue Borama Hardware build from Phase N, Step X. Completed: [list what passed checkpoint].

Reference documents: PRD, App Flow, Tech Stack, Guidelines, Schema (all PDFs in project folder).'

Claude Code will resume from exactly where you left off.